

Lecture Summary

Main Points

- Templates are HTML files enhanced by Django syntax.
- Views send context data to templates through render().
- Variables, loops, and conditions make pages dynamic.
- Inheritance and include reduce repetition.
- Static files and URL tags complete real page building.

Classroom Connection

If students understand this slide, they are ready for models-driven templates.

Teaching Note

Use this slide as your final recap.

Lecture 7: Django Templates and Dynamic Pages

Main Themes

- Templates and render()
-
- Variables, loops, and conditions
-
- Inheritance, include, and static files
-
- Hands-on mini project and classroom exercises

Teaching Promise

Every major idea today is tied to a practical web page example so you can speak from the slide with confidence.

Classroom Hint

Start with the student portal and keep returning to it throughout the lecture.

Lecture 7: Django Templates and Dynamic Pages

Lecture Goals

- Understand what a Django template is and why we use it.
- Learn how `render()` connects a view with an HTML file.
- Send variables, lists, and condition results from the view to the template.
- Use template tags such as `for`, `if`, `block`, `extends`, and `include`.
- Build pages that feel dynamic instead of hard-coded.

Real-life link

Think of a student portal page that shows a different name and course list for each student.

Teaching Note

Tell students that this lecture is the bridge between plain HTML and real web applications.

Why Templates Matter

Main Idea

- A static HTML page shows the same content every time.
- A template allows placeholders that Django fills with data.
- One template can serve many users and many records.
- Templates reduce repetition and improve maintainability.

Classroom Connection

A single course page layout can display different course titles, codes, and instructors.

Teaching Note

Emphasize the saving in time and effort.

Static HTML vs Dynamic Template

Static HTML

- Fixed text inside the file.
- No placeholders for changing data.
- Good for a simple one-page site.
- Bad choice for a student system or e-commerce site.

Django Template

- Contains placeholders like `{{ name }}`.
- Receives values from the view.
- Can loop through a list of records.
- Can show or hide sections with conditions.

What Is a Template in Django?

Main Points

- It is still HTML, so the browser can display it.
- It contains variables: `{{ variable_name }}`
- It contains tags: `{% for %}`, `{% if %}`, `{% extends %}`
- It may contain filters: `{{ name|upper }}`
- Django renders it before sending it to the browser.

Classroom Connection

The browser never sees the Django tags; it only receives the final HTML output.

Teaching Note

Repeat this sentence: the template is processed on the server, not in the browser.

Where Do We Store Templates?

Main Points

- Common structure:
`app_name/templates/app_name/file.html`
- This avoids name collisions between apps.
- Some projects use a global templates folder at the project level.
- You must tell Django where to find templates in settings if needed.

Classroom Connection

`students/templates/students/list.html` is safer than `list.html` alone.

Teaching Note

Mention that organization now prevents confusion later.

```
TEMPLATES = [  
    {  
        'BACKEND':  
        'django.template.backends.django.DjangoTemplate  
s',  
        'DIRS': [],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
  
        'django.template.context_processors.request',  
  
        'django.contrib.auth.context_processors.auth',  
  
        'django.contrib.messages.context_processors.mes  
sages',  
            ],  
        },  
    },  
]  
]
```

```
from pathlib import Path
```

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

```
TEMPLATES = [  
    {  
        'BACKEND':  
        'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
  
            'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    },  
]  
]
```

project-level templates folder


```
import os  
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
from pathlib import Path  
BASE_DIR = Path(__file__).resolve().parent.parent
```

First Template File

home.html

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Students Home</title>
5 </head>
6 <body>
7     <h1>Welcome to the Students App</h1>
8     <p>This page is rendered by Django.</p>
9 </body>
10 </html>
```

What to explain

Explain that this is normal HTML. At this stage nothing is dynamic yet.

Ask students

If we open this page ten times, will it change?

Common mistake

Students may save the file in the wrong folder.

Using render() in a View

views.py

```
1 from django.shortcuts import render
2
3 def home(request):
4     return render(request, 'students/home.html')
```

What to explain

Show that render() takes the request and the template path. It creates the response automatically.

Ask students

What does Django send back to the browser here?

Common mistake

Students may forget the correct relative template path.

URL to Template Flow

Request -> URL -> View -> Template -> Response

urls.py

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.home, name='home'),
6 ]
```

What to explain

Remind students that the URL calls the view, and the view renders the template.

Ask students

Which file receives the browser request first: template or URLconf?

Common mistake

Students may think the template is opened directly without a view.

Template Variables

Variables display changing values.

Main Points

- Use double curly braces: `{{ variable_name }}`
- The variable name must come from the context dictionary in the view.
- If the variable is missing, Django usually shows empty output.
- Variables can be strings, numbers, objects, or object attributes.

Classroom Connection

`{{ student.name }}` works if student is an object passed to the template.

Teaching Note

Write the syntax on the board: two curly braces for output.

Sending a Variable from the View

Context is a dictionary of names and values.

views.py

```
1 from django.shortcuts import render
2
3 def welcome(request):
4     context = {
5         'student_name': 'Ali',
6         'level': 2,
7     }
8     return render(request, 'students/welcome.html', context)
```

What to explain

Explain the context dictionary: keys are names used in the template, values are actual data.

Ask students

What will the key name be inside the HTML file?

Common mistake

Students may confuse the dictionary key with the Python variable name.

Displaying Variables in HTML

The template reads names from the context.

welcome.html

```
1 <h1>Welcome {{ student_name }}</h1>
2 <p>Your academic level is {{ level }}</p>
```

What to explain

Show how Django replaces each placeholder with the real value before the browser sees it.

Ask students

What final HTML will the browser receive for Ali?

Common mistake

Students may place extra spaces or wrong braces.

Why Lists Matter in Templates

Most real pages display multiple records, not one value only.

Main Points

- Student lists
- Course lists
- Messages, notifications, and comments
- Search results and tables

Classroom Connection

A university system might show all registered courses, not one course only.

Teaching Note

Prepare students for loops.

Sending a List to the Template

A list can be passed in the context just like a string.

views.py

```
1 def course_list(request):
2     courses = ['Python', 'Database', 'AI', 'Networks']
3     return render(request, 'students/courses.html',
{'courses': courses})
```

What to explain

The key courses is linked to a Python list.
The template will iterate over it.

Ask students

If the list has four items, how many list elements do you expect on the page?

Common mistake

Students may think Django prints the whole Python list automatically in a nice layout.

Template for Loop

Use {% for item in items %} ... {% endfor %}

courses.html

```
1 <h2>Registered Courses</h2>
2 <ul>
3     {% for course in courses %}
4         <li>{{ course }}</li>
5     {% endfor %}
6 </ul>
```

What to explain

The for tag repeats the HTML block once for every item in the list.

Ask students

What would happen if the list had zero items?

Common mistake

Students may forget {% endfor %}.

Handling an Empty List

Good pages handle missing data gracefully.

`courses.html`

```
1 <h2>Registered Courses</h2>
2 {% if courses %}
3     <ul>
4         {% for course in courses %}
5             <li>{{ course }}</li>
6         {% endfor %}
7     </ul>
8 {% else %}
9     <p>No courses found.</p>
10 {% endif %}
```

What to explain

Use if to decide whether to show a list or a fallback message.

Ask students

Why is this better than showing an empty area?

Common mistake

Students may nest tags incorrectly or forget endif.

Template if Tag

Conditional rendering means showing different HTML in different situations.

Main Points

- Syntax starts with {% if condition %}
- You may use {% elif %} and {% else %}
- Always close the condition with {% endif %}
- Useful for login state, marks, status messages, and empty records.

Classroom Connection

Show “Passed” if mark $\geq$ passing value, otherwise show “Try again”.

Teaching Note

Link to Python if statement from Lecture 2.

Example: Pass or Fail Message

Templates can show different messages based on context values.

result.html

```
1 <h2>Student Result</h2>
2 {% if mark >= 50 %}
3     <p>Passed</p>
4 {% else %}
5     <p>Failed</p>
6 {% endif %}
```

What to explain

Explain that the value of mark comes from the view. The template only decides which branch to display.

Ask students

If mark is 48, which paragraph will appear?

Common mistake

Students may assume heavy business logic should live here.

Logic in the View vs Logic in the Template

Keep templates simple and readable.

Good in the view

- Complex calculations
- Database queries
- Decision-heavy business rules
- Preparing context data

Good in the template

- Simple display conditions
- Loops for rendering lists
- Formatting output
- Choosing visible sections

Displaying Object Attributes

Templates can access attributes with dot notation.

profile.html

```
1 <h2>Student Profile</h2>
2 <p>Name: {{ student.name }}</p>
3 <p>Level: {{ student.level }}</p>
4 <p>Department: {{ student.department }}</p>
```

What to explain

If student is an object or model instance, dot notation accesses its attributes.

Ask students

Does student here refer to one object or a list of objects?

Common mistake

Students may expect Python method calls with parentheses in templates.

Looping Over Objects

Each iteration can display object attributes.

students.html

```
1 <ul>
2     {% for student in students %}
3         <li>{{ student.name }} - Level {{ student.level
4     }}</li>
5     {% endfor %}
6 </ul>
```

What to explain

Each student inside the loop is one object from the students collection.

Ask students

What would one line look like for a student named Mona in level 2?

Common mistake

Students may use the same name for the list and the loop variable.

Template Filters

Filters transform output values without changing the original data.

Main Points

- `{{ name|upper }}` -> uppercase
- `{{ name|lower }}` -> lowercase
- `{{ text|length }}` -> number of characters or items
- `{{ title|default:"No Title" }}` -> fallback value
- `{{ name|title }}` -> title case

Classroom Connection

Filters are for presentation, not business logic.

Teaching Note

Write the pipe symbol on the board and say “pipe means pass through a filter.”

What is a Django Template Filter?

Quick explanation before the detailed list

A template filter formats or transforms a value before displaying it in HTML.

It is used inside Django templates, not in Python code directly.

Filters are ideal for simple presentation logic such as making text uppercase, formatting dates, truncating long text, or showing default values.

For complex logic, calculations, and database work, use views/models instead of templates.

Examples

```
{{ name|upper }}
```

```
{{ bio|truncatewords:10 }}
```

```
{{ created_at|date:"d/m/Y" }}
```

```
{{ email|default:"No email" }}
```

Text Filters (Part 1)

Common text filters students use first

Filter	What it does	Example	Output / Note
upper	Converts text to uppercase letters	{{ name upper }}	mohamed → MOHAMED
lower	Converts text to lowercase letters	{{ name lower }}	Mohamed → mohamed
title	Capitalizes the first letter of each word	{{ department title }}	computer science → Computer Science
capfirst	Capitalizes only the first letter of the first word	{{ text capfirst }}	hello world → Hello world
length	Returns number of letters or list items	{{ students length }}	Useful for counts

Text Filters (Part 2)

Useful for long descriptions and missing values

Filter	What it does	Example	Output / Note
default	Shows a fallback value if empty	<code>{{ nickname default:"Guest" }}</code>	If empty → Guest
default_if_none	Shows fallback only if value is None	<code>{{ value default_if_none:"No value" }}</code>	Works only for None
truncatechars	Shortens text after a certain number of characters	<code>{{ text truncatechars:20 }}</code>	Cuts long text and adds ...
truncatewords	Shortens text after a certain number of words	<code>{{ text truncatewords:10 }}</code>	Useful in previews
cut	Removes all occurrences of given text/character	<code>{{ text cut:" " }}</code>	Can remove spaces

Safety and Formatting Filters

Important note: safe can create security issues if content is untrusted

Filter	What it does	Example	Output / Note
safe	Treats text as safe HTML	{{ content safe }}	Use carefully for trusted HTML only
escape	Escapes HTML and shows it as normal text	{{ content escape }}	Prevents HTML rendering
linebreaks	Converts line breaks into HTML paragraphs and 	{{ text linebreaks }}	Good for long user text
linebreaksbr	Converts line breaks to only	{{ text linebreaksbr }}	Simpler than linebreaks
filesizeformat	Formats file size into KB/MB/etc.	{{ file.size filesizeformat }}	1024 → 1 KB

Date and Time Filters

Very common in news, events, and timestamps

Filter	What it does	Example	Output / Note
date	Formats a date value	{{ mydate date:"Y-m-d" }}	Example: 2026-08-03
time	Formats a time value	{{ mytime time:"H:i" }}	Example: 14:30
timesince	Shows time since a past date	{{ created_at timesince }}	2 days / 3 hours
timeuntil	Shows time until a future date	{{ exam_date timeuntil }}	Useful for countdowns

List and Sequence Filters

Useful in lists, cards, menus, and quick displays

Filter	What it does	Example	Output / Note
first	Returns the first item in a list	<code>{{ students first }}</code>	Useful with arrays/lists
last	Returns the last item in a list	<code>{{ students last }}</code>	Gets final element
join	Joins list items into one string using a separator	<code>{{ courses join:", " }}</code>	Python, AI, DB
slice	Returns part of a list or string	<code>{{ students slice:"3" }}</code>	First 3 items
random	Returns a random item from a list	<code>{{ quotes random }}</code>	Changes each refresh
dictsort	Sorts dictionaries/objects by a key	<code>{{ students dictsort:"name" }}</code>	Sort by field

Numeric and Logical Filters

Good for reports, statistics, and quantities

Filter	What it does	Example	Output / Note
yesno	Displays words instead of True/False	<code>{{ is_active yesno:"Yes,No" }}</code>	True → Yes, False → No
add	Adds a value	<code>{{ age add:"5" }}</code>	20 → 25
pluralize	Adds plural suffix when count is not 1	<code>{{ count }} item{{ count pluralize }}</code>	1 item / 2 items
floatformat	Controls decimal places	<code>{{ price floatformat:2 }}</code>	12.3456 → 12.35

Practice Examples

Short exercises for students

1) Convert a user name to uppercase.

Answer: `{{ username|upper }}`

2) Show first 15 words from a long description.

Answer: `{{ description|truncatewords:15 }}`

3) Show a default message if email is missing.

Answer: `{{ email|default:"No email" }}`

4) Format an event date as day/month/year.

Answer: `{{ event_date|date:"d/m/Y" }}`

5) Show Yes/No instead of True/False.

Answer: `{{ is_active|yesno:"Yes,No" }}`

6) Show only first 3 students from a list.

Answer: `{{ students|slice:"3" }}`

7) Show a number with 2 decimal places.

Answer: `{{ gpa|floatformat:2 }}`

Best Filters to Memorize First

Recommended starter list for students

Text: upper, lower, title, capfirst

Defaults: default, default_if_none

Shortening: truncatechars, truncatewords

Lists: length, first, last, join, slice

Dates: date, time, timesince

Numbers/logic: yesno, add, floatformat, pluralize

Safety: safe, escape

Summary

What students should remember

A Django template filter changes the display of a value inside templates.

It is written using the pipe symbol | .

Many filters are used for text formatting, dates, lists, numbers, and safe output.

Use filters for simple presentation logic only.

Do not move heavy logic to templates; keep it in views/models.

Examples of Common Filters

Use filters to polish the page output.

filters demo

```
1 <p>{{ student_name|upper }}</p>
2 <p>{{ department|title }}</p>
3 <p>{{ bio|length }}</p>
4 <p>{{ nickname|default:"Guest" }}</p>
```

What to explain

Explain each filter one by one and show expected output.

Ask students

Which filter would you use if nickname is empty?

Common mistake

Students may forget the colon syntax in some filters.

Template Comments

Comments help developers without appearing in the browser.

comments

```
1 {% comment %}  
2 This section is for the course table.  
3 It may later become a reusable include.  
4 {% endcomment %}
```

What to explain

Django comments are removed during rendering and are not visible in page source as final content.

Ask students

Why are comments useful in team projects?

Common mistake

Students may think HTML comments and template comments are identical in behavior.

The Problem of Repetition

Many pages share the same header, navbar, and footer.

Main Points

- If every page duplicates the full layout, changes become slow.
- One small edit may need to be repeated in ten files.
- Repetition increases mistakes and inconsistency.
- Django template inheritance solves this elegantly.

Classroom Connection

Imagine editing the navbar link in 15 pages one by one.

Teaching Note

Set up the need for base.html.

Base Template Structure

A parent template defines common layout.

base.html

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>{% block title %}My Site{% endblock %}</title>
5 </head>
6 <body>
7     <header>Faculty Portal</header>
8     <nav>Home | Courses | Students</nav>
9
10    {% block content %}
11    {% endblock %}
12
13    <footer>All rights reserved</footer>
14 </body>
15 </html>
```

What to explain

Explain the idea of blocks: child templates replace specific sections while the shared layout stays the same.

Ask students

Which parts stay common for all pages here?

Common mistake

Students may forget that the parent file itself is usually not the page-specific content.

Child Template with extends

A child template fills the blocks defined in base.html.

home.html

```
1 {% extends 'students/base.html' %}
2
3 {% block title %}Home{% endblock %}
4
5 {% block content %}
6     <h1>Welcome to the Home Page</h1>
7     <p>This content is specific to this page.</p>
8 {% endblock %}
```

What to explain

Show that extends must appear near the top. The child page only writes the changing content.

Ask students

Why is this shorter than writing a full HTML page?

Common mistake

Students may type content outside the block or use wrong path in extends.

Rules of Template Inheritance

These rules prevent common errors.

Main Points

- Create meaningful block names such as title, content, sidebar.
- Put `{% extends %}` at the top of the child template.
- Keep the base template general and reusable.
- Do not fill blocks in the base with page-specific details.

Classroom Connection

Think of `base.html` as the building structure and child templates as decorated rooms.

Teaching Note

Use the building analogy.

Using include for Reusable Pieces

include inserts a smaller template inside another template.

example

```
1 <!-- navbar.html -->
2 <nav>
3     <a href="/">Home</a>
4     <a href="/students/">Students</a>
5 </nav>
6
7 <!-- base.html -->
8 <body>
9     {% include 'students/navbar.html' %}
10    {% block content %}{% endblock %}
11 </body>
```

What to explain

Use include for components like navbars, alerts, cards, and tables.

Ask students

When is include better than extends?

Common mistake

Students may confuse parent-child inheritance with inserting small reusable parts.

Templates and Static Files

HTML alone is not enough for visual design.

Main Points

- CSS styles the page.
- JavaScript adds browser-side behavior.
- Images and icons improve clarity.
- Django uses the static system to serve these files in development.

Classroom Connection

A template controls structure; CSS controls appearance.

Teaching Note

Clarify the separation of concerns.

Loading Static Files in a Template

Use the static template tag.

base.html

```
1 {% load static %}
2 <!DOCTYPE html>
3 <html>
4 <head>
5     <link rel="stylesheet" href="{% static
'students/style.css' %}">
6 </head>
7 <body>
8     
9 </body>
10 </html>
```

What to explain

You must load static first. Then Django can build the correct static file URL.

Ask students

Why do we not type the full file path from the computer?

Common mistake

Students may forget {% load static %}.

Linking Pages with the url Tag

Avoid hard-coded links when possible.

template link

```
1 <a href="{% url 'home' %}">Home</a>  
2 <a href="{% url 'student_list' %}">Students</a>
```

What to explain

The url tag uses the route name from urls.py. This is safer than writing literal paths.

Ask students

What happens if the path changes but the route name stays the same?

Common mistake

Students may forget to name their URLs in urls.py.

url Tag with Parameters

Dynamic pages often need an ID or slug.

student row

```
1 <a href="{% url 'student_detail' student.id %}">
2     View Profile
3 </a>
```

What to explain

The template passes the current student.id into the URL pattern.

Ask students

If student.id is 7, what kind of path do you expect?

Common mistake

Students may not match the URL parameters required in urls.py.

مثال	الوظيفة	Tag
<code>{% extends 'base.html' %}</code>	يرث من template أساسي	<code>{% extends %}</code>
<code>{% block content %}</code>	يحدد جزء يمكن استبداله	<code>{% block %}</code>
<code>{% endblock %}</code>	نهاية الـ block	<code>{% endblock %}</code>
<code>{% include 'navbar.html' %}</code>	إدخال template داخل template أخرى	<code>{% include %}</code>
<code>{% if user.is_authenticated %}</code>	شرط	<code>{% if %}</code>
<code>{% elif marks > 50 %}</code>	شرط إضافي	<code>{% elif %}</code>
<code>{% else %}</code>	الحالة البديلة	<code>{% else %}</code>
<code>{% endif %}</code>	نهاية الشرط	<code>{% endif %}</code>
<code>{% for student in students %}</code>	Loop للتكرار	<code>{% for %}</code>
<code>{% empty %}</code>	ينفذ إذا كانت القائمة فارغة	<code>{% empty %}</code>
<code>{% endfor %}</code>	نهاية الـ loop	<code>{% endfor %}</code>

{% comment %} text {% endcomment %}	تعليق لا يظهر للمستخدم	{% comment %}
{% endcomment %}	نهاية التعليق	{% endcomment %}
{% load static %}	تحميل static files	{% load static %}
{% static 'css/style.css' %}	الوصول لملف static	{% static %}
{% csrf_token %}	حماية الفورم من CSRF	{% csrf_token %}
{% url 'home' %}	إنشاء URL ديناميكي	{% url %}
{% with total=students %}	إنشاء متغير مؤقت	{% with %}
{% endwith %}	نهاية with	{% endwith %}
{% autoescape off %}	تشغيل/إيقاف escaping	{% autoescape %}

<code>{% endautoescape %}</code>	نهاية autoescape	<code>{% endautoescape %}</code>
<code>{% cycle 'red' 'blue' %}</code>	التبديل بين قيم متعددة	<code>{% cycle %}</code>
<code>{% firstof name username 'Guest' %}</code>	أول قيمة غير فارغة	<code>{% firstof %}</code>
<code>{% now 'Y-m-d' %}</code>	عرض التاريخ/الوقت الحالي	<code>{% now %}</code>
<code>{% spaceless %}</code>	إزالة المسافات بين HTML tags	<code>{% spaceless %}</code>
<code>{% endspaceless %}</code>	نهاية spaceless	<code>{% endspaceless %}</code>
<code>{% regroup students by department as grouped %}</code>	تجميع البيانات حسب قيمة	<code>{% regroup %}</code>
<code>{% verbatim %}</code>	منع Django من تفسير الكود	<code>{% verbatim %}</code>
<code>{% endverbatim %}</code>	نهاية verbatim	<code>{% endverbatim %}</code>
<code>{% lorem 3 p %}</code>	إنشاء نص تجريبي	<code>{% lorem %}</code>
<code>{% filter upper %}</code>	تطبيق filter على block كامل	<code>{% filter %}</code>
<code>{% endfilter %}</code>	نهاية filter	<code>{% endfilter %}</code>
<code>{% debug %}</code>	عرض معلومات debug	<code>{% debug %}</code>
<code>{% widthratio 5 10 50 %}</code>	حساب نسبة رقمية	<code>{% widthratio %}</code>
<code>{% templatetag openblock %}</code>	عرض رموز template نفسها	<code>{% templatetag %}</code>

Data Flow Review

The server prepares, the template displays, the browser receives.

View responsibilities

- Collect data
- Query the database
- Build the context dictionary
- Choose the template to render

Template responsibilities

- Display variables
- Repeat HTML with loops
- Show or hide sections
- Format output with filters

Mini Project Goal for This Lecture

Build a simple student list page with a base template.

Main Points

- Create base.html
- Create students/list.html
- Pass a list of student dictionaries from the view
- Display the students inside a loop
- Show a “No students found” message if the list is empty

Classroom Connection

This mini project brings together render, context, loops, and inheritance.

Teaching Note

Tell them this is the practical heart of the lecture.

Mini Project View

Pass a list of dictionaries from Python to the template.

views.py

```
1 from django.shortcuts import render
2
3 def student_list(request):
4     students = [
5         {'name': 'Ali', 'level': 2},
6         {'name': 'Mona', 'level': 2},
7         {'name': 'Omar', 'level': 3},
8     ]
9     return render(request, 'students/list.html', {'students':
students})
```

What to explain

Each dictionary acts like one student record. Later this list can come from the database.

Ask students

How many student cards or list items should appear?

Common mistake

Students may use curly braces incorrectly or forget commas between dictionaries.

Mini Project Template

Render each student neatly.

list.html

```
1 {% extends 'students/base.html' %}
2
3 {% block title %}Students{% endblock %}
4
5 {% block content %}
6     <h2>Students List</h2>
7     <ul>
8         {% for student in students %}
9             <li>{{ student.name }} - Level {{ student.level
10 }}</li>
11         {% empty %}
12             <li>No students found.</li>
13         {% endfor %}
14     </ul>
15 {% endblock %}
```

What to explain

Introduce the useful {% empty %} option inside a for loop. It handles empty lists elegantly.

Ask students

What is better here: using if around the loop or using empty inside the loop?

Common mistake

Students may forget that empty belongs inside the for block.

Simple CSS for the Student List

Use CSS to make the page easier to read.

style.css

```
1 body {  
2     font-family: Arial, sans-serif;  
3 }  
4  
5 li {  
6     margin-bottom: 8px;  
7     padding: 6px;  
8     background-color: #f4f7fb;  
9 }
```

What to explain

This is a quick reminder that templates work well with external CSS.

Ask students

Does CSS make the page dynamic by itself?

Common mistake

Students may put CSS in the wrong static folder.

Displaying Data in a Table

Tables are common for admin panels and management systems.

table example

```
1 <table border="1">
2     <tr>
3         <th>Name</th>
4         <th>Level</th>
5     </tr>
6     {% for student in students %}
7     <tr>
8         <td>{{ student.name }}</td>
9         <td>{{ student.level }}</td>
10    </tr>
11    {% endfor %}
12 </table>
```

What to explain

Show the difference between list view and table view. Same data, different presentation.

Ask students

Which layout is better for many columns of data?

Common mistake

Students may place the loop around the entire table incorrectly.

Common Template Errors

Most mistakes are small, but they stop the page from rendering correctly.

Main Points

- Wrong template path in render()
- File saved in the wrong templates folder
- Missing {% endfor %} or {% endif %}
- Using a variable name that is not in the context
- Forgetting {% load static %}

Classroom Connection

A small missing brace can break the page.

Teaching Note

Normalize mistakes so students are not afraid of them.

How to Debug a Template Problem

Use a simple checklist before asking for help.

Main Points

- Check the file path first.
- Check the context keys in the view.
- Check braces and closing tags.
- Read the traceback carefully.
- Test with a simpler version of the template.

Classroom Connection

Debugging is a skill, not a punishment.

Teaching Note

Tell them to isolate the problem.

Class Activity 1: Predict the Output

Read the code mentally before running it.

Main Points

- Given a context with name = "Mona" and level = 2
- Predict the result of: `<p>{{ name }}</p>`
- Predict the result of: `{% if level == 2 %}<p>Second Year</p>{% endif %}`
- Predict the result of a loop over `["AI", "DB"]`

Classroom Connection

Ask students to answer before you reveal the result.

Teaching Note

Use the board and let two students write their predictions.

Class Activity 2: Fix the Template Errors

Students identify and correct mistakes.

broken template

```
1 <ul>
2 {% for student in students %}
3     <li>{{ student.name }} - Level {{ student.level }}</li>
4 {% endfor %}
5 </ul>
```

What to explain

There is a missing brace and an unclosed tag syntax error. Let students find both.

Ask students

Who can point to the exact broken symbols?

Common mistake

Do not fix it immediately; wait for students.

Class Activity 3: Design a base Template

Students think structurally, not only syntactically.

Main Points

- Suggest which parts should go into base.html.
- Suggest which parts belong only to one page.
- Decide good block names.
- Discuss whether navbar should be part of base or include.

Classroom Connection

This activity trains architectural thinking.

Teaching Note

Accept multiple reasonable answers.

Connection to Upcoming Lectures

Templates become much more powerful with models and forms.

Main Points

- With models, the list comes from the database instead of hard-coded data.
- With forms, the template can display input fields and validation messages.
- With the admin panel, you can add data and then see it on templates.

Classroom Connection

Today we simulate data manually. Soon the data will be real.

Teaching Note

Build motivation for Lecture 8 and 9.

Template Best Practices

Small habits make a big difference in real projects.

Main Points

- Use clear variable names in the context.
- Keep templates simple and readable.
- Prefer inheritance over copy-paste.
- Organize files by app.
- Comment tricky sections when needed.

Classroom Connection

Readable templates help teams move faster.

Teaching Note

Relate this to software engineering, not just Django syntax.

Running the Project to Test Templates

Remind students of the command flow.

terminal

```
1 python manage.py runserver
```

What to explain

After saving the view, URL, and template files, run the server and test in the browser.

Ask students

What should we do if the server is already running and we changed files?

Common mistake

Students may edit files but forget to refresh the browser.

Lab Checklist for Students

A simple order prevents confusion in the lab.

Main Points

- Open the project folder in VS Code.
- Confirm the app exists and is added in INSTALLED_APPS.
- Create the templates folder in the right place.
- Write the view, then the URL, then the HTML file.
- Run the server and test one page at a time.

Classroom Connection

One page working well is better than five broken pages.

Teaching Note

Encourage incremental testing.

Quick Quiz 1

Answer verbally before we move on.

Main Points

- What function usually returns a rendered template?
- What syntax prints a variable value?
- What tag repeats HTML for each item in a list?
- What tag creates parent-child template inheritance?

Classroom Connection

Wait for students to answer, then confirm.

Teaching Note

Do not rush through quiz slides.

Quick Quiz 2

Check understanding of structure and debugging.

Main Points

- Why is base.html useful?
- When would you use include?
- Why is hard-coded linking weaker than `{% url %}`?
- Give one common template error.

Classroom Connection

Use student answers as revision, not judgment.

Teaching Note

Invite short explanations, not one-word answers only.

Homework Task

Small but complete practice after class.

Main Points

- Create a base template with header and footer.
- Create a page called courses.html that extends the base.
- Pass a list of at least four courses from the view.
- Display the list using a loop.
- Show a message if the list is empty.
- Add one CSS file through the static system.

Classroom Connection

This homework mirrors the lecture workflow.

Teaching Note

Tell them to submit screenshots and code files if required.

Template Syntax Cheat Sheet

Deepening understanding through focused revision.

Main Points

- `{{ variable }}` prints a value.
- `{% tag %}` performs logic or structure.
- `{# comment #}` or `{% comment %}` hides notes for developers.
- Use `|` for filters such as `upper` and `length`.

Classroom Connection

Keep this slide as a revision anchor.

Teaching Note

Pause and let students speak.

Naming Context Variables Clearly

Deepening understanding through focused revision.

Main Points

- Bad: data, item1, x
- Better: student_name, course_list, total_marks
- Clear names make templates readable.

Classroom Connection

Relate to clean code from Python basics.

Teaching Note

Pause and let students speak.

Why the Browser Never Sees Django Tags

Deepening understanding through focused revision.

Main Points

- Django processes templates on the server.
- The browser only receives final HTML, CSS, and JS.
- This is why users cannot directly execute Django template tags.

Classroom Connection

Contrast with JavaScript running in the browser.

Teaching Note

Pause and let students speak.

TemplateDoesNotExist Error

Deepening understanding through focused revision.

Main Points

- Usually caused by wrong file path or folder structure.
- Check TEMPLATES setting and app templates folder.
- Read the missing path shown in the traceback.

Classroom Connection

A useful debugging slide.

Teaching Note

Pause and let students speak.

Using `{% empty %}` in a for Loop

Deepening understanding through focused revision.

Main Points

- `{% empty %}` is cleaner than a separate if in some cases.
- It keeps the loop and fallback message together.
- Very useful for list pages.

Classroom Connection

Show students both patterns and compare.

Teaching Note

Pause and let students speak.

When to Use include vs extends

Deepening understanding through focused revision.

Main Points

- extends = full parent-child page structure
- include = insert a small reusable partial
- Many projects use both together.

Classroom Connection

Use navbar and footer examples.

Teaching Note

Pause and let students speak.

Design Question for Students

Deepening understanding through focused revision.

Main Points

- What blocks would you create for an academic portal base template?
- Possible answers: title, content, sidebar, scripts

Classroom Connection

Pause for brainstorming.

Teaching Note

Pause and let students speak.

Mini Reflection

Deepening understanding through focused revision.

Main Points

- What felt easier today: variables, loops, or inheritance?
- What needs more practice in the lab?

Classroom Connection

Use this before closing.

Teaching Note

Pause and let students speak.